

Agentic Code Intelligence

One of the main use-cases of AI agents is to analyze and understand existing codebases. Getting to the right files and the right blocks of code remains a bottleneck for fixing bugs and adding new features, and it only compounds as the codebase grows.

What is this problem all about?

The problem at its core is: code retrieval.

Given a library of code and a query in natural language, provide a ranking of the code snippets in order of their relevance to the query.

For example, if the query is *How is the input preprocessed before going to the main function?* and you have the following three snippets:

<pre>// Code#1 function normalize(str) { const str2 = str.trim(); return forward(str2); }</pre>	<pre>// Code#2 function check(s) { var pre = s.slice(0,6); return pre === 'en-US'; }</pre>	<pre>// Code#3 function perf(str) { if (act(A, str)) { return act(B, str) } }</pre>
---	--	---

Your ranking should be something like Code#1 > Code#2 > Code#3. This is a pretty difficult problem and likely cannot be solved in a single pass. You should be aware that the actual code snippets range in the thousands and the code snippets can also increase in length.

What does the solution look like?

We are looking for a solution that improves the *retrieval*, which is the part explained before. Generating an answer for the query, explaining the results or anything to do with the generation that takes place after the retrieval is **out of scope** for this problem statement.

Your solution can have improvements based around (but not limited to):

1. Categorizing the query
2. Pre-processing the query
3. Categorizing the retrieved code snippets
4. Pre/post processing of the code snippets
5. Performing multiple retrieval passes based on the above

You are free to use the models and methods available and combine them with your own methods so that your solution retrieves the most relevant code snippets for a given query.

Most retrieval and embedding models are small and fast enough to run on CPU alone. As such, your solution is also expected to run on CPU with minimal GPU resource utilization.

Submission Goals

These are the goals of the solutions, in the order of their importance

P0: Retrieval Accuracy

This is the base criteria, your solution should retrieve the relevant code snippets in response to a query. This is what the example above described.

P1: Retrieval across versions

Code-bases are hardly static, they keep changing with new commits added every minute. Your solution should support performing retrieval on *different versions* of the code snippets.

This means that your solution should be able to rebuild any indexes, caches, etc for any version/change in a reasonable amount of time.

Bonus: Evolutionary Retrieval

This is an extension of P1. If your solution is able to support different versions of the code snippets, you should be able to retrieve code snippets *across all versions* of them. This has specific implications for retrieval, because even across different versions, the snippets would still be very similar, which would make them hard to rank properly.

Why don't I give an LLM the query and the snippets and ask it to rank?

The number of snippets and their length are too long to fit in the context window of any LLM. Also, LLMs are slow when it comes to processing long texts, retrieval is generally the first step in a RAG pipeline and is expected to be faster than the generation step, which involves an LLM.

How is the solution evaluated?

We will evaluate your submissions in two stages:

Screening

We will rank your solutions on the P0 submission goal (Retrieval Accuracy) based on their performance on the test split of the CoIR apps dataset¹. You will have to submit a csv file with the responses from the test split of the dataset, from which we will measure:

1. NDCG@10
2. MRR

You can measure these metrics yourself to self-evaluate your solutions against well-known approaches or your friends. This will be a competitive screening meaning we will be selecting the top submissions for hands-on evaluation.

How to run your solution on the test split

You should use the MTEB library to simply running the evaluation. The following code demonstrates how to use the MTEB library to generate the evaluation JSON.

```
import numpy as np
from sentence_transformers import SentenceTransformer

import mteb
from mteb.models.abs_encoder import AbsEncoder
from mteb.models.model_meta import ModelMeta
from mteb.types import PromptType

class PrePostPipelineEncoder(AbsEncoder):
    # Your implementation here

def main() -> None:
    model = PrePostPipelineEncoder()
```

¹<https://huggingface.co/datasets/CoIR-Retrieval/apps/viewer/default/test>

```

task = mteb.get_task("AppsRetrieval") # Make sure you choose this task
result = mteb.evaluate(
    model,
    [task],
    encode_kwargs={"batch_size": 64},
)

# Write the evaluation JSON you asked for.
task_result = list(result.task_results)[0]
with open("appsretrieval_results.json", "w") as f: # Upload this file
    json.dump(task_result.to_dict(), f, indent=2)

```

You can look at the MTEB library² for more details.

At the time of submission you should make a GitHub release and upload the generated file.

Hands-on

During the hands-on evaluation, we will review your PPT, demo video along with running your code on certain type of queries (which will be similar to that of the dataset). We will also evaluate the P1 (Retrieval across versions) and the Bonus submission goals here.

What to know before submitting?

Please make sure you have ran your solution's inference on the test split of the CoIR apps dataset. You should have the following ready:

1. json file with the inference on the test split on the CoIR apps dataset
2. PPT
3. GitHub repo with instructions on how to run your submission

You should make sure that your submission can be run by following the steps in the GitHub repo. Please attach any files needed as artifacts as a *Release* in the GitHub repo.

How to submit the JSON file

You should already have JSON file containing the results of inference on the test split. You should follow the GitHub guide³ to create a release on your GitHub repo and upload the JSON file obtained from running the MTEB evaluation as demonstrated above.

How do I submit the json files? What is the format of the json file?

You should generate the JSON files from the MTEB library, it will take care of the format. Once you have the file, you can follow the section "How to submit the JSON file" above to create a GitHub release and upload your JSON file.

²https://docs.mteb.org/get_started/usage/running_the_evaluation/

³<https://docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository#creating-a-release>

What should I put in the PPT?

Our PPT template answers this pretty well but we would be interested in seeing the details of your approach (which pre/post processing steps, which embedding model, etc). You could also show the retrieval results for a tough query which shows how well your solution is working.

What should the demo show?

We would like to see the solution working in the demo. You should not show us just the inference results or the numbers. You should show the responses for a given query. We would also be interested in seeing how fast your solution is.